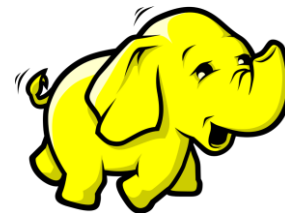


TD7: Ingeniería de Datos

Procesamiento de datos

Hadoop como opción para almacenar el Master dataset

— — —



Componentes principales

- ❖ HDFS (Hadoop Distributed File System)
- ❖ MapReduce

Recordemos ¿qué problema intentaba resolver Hadoop?

- Los datos ya no entran en una sola computadora.
- Comprar una supercomputadora es muy costoso.
- Se distribuyen datos y cómputo en muchas máquinas comunes.
- El sistema tolera fallas de hardware.

MapReduce

— — —



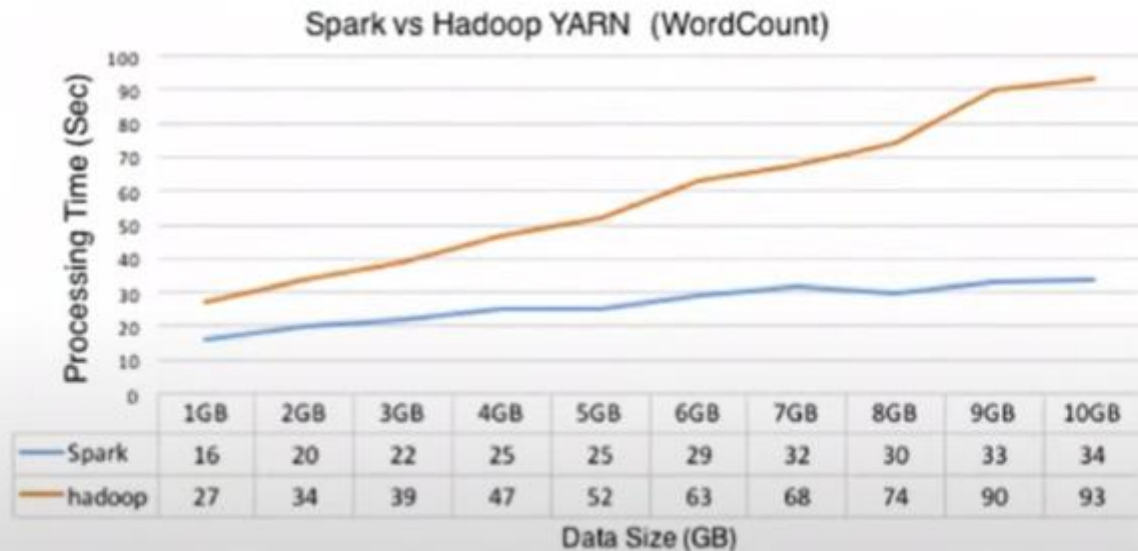
- ❖ Dividir el trabajo en secuencias de *maps* y *reduces*.
- ❖ **Pensar el problema de alto nivel.**
- ❖ No pensar cómo se resuelven los detalles técnicos del paralelismo y la distribución.

Lo único que necesitamos explicarle a MapReduce es qué es lo que queremos computar y cómo se calcula.

Introducción a Apache Spark



- Se desarrolló inicialmente en Berkeley
- Sustituye a Hadoop Mapreduce



Spark

— — —



- ❖ Diseñado para procesamiento distribuido de propósito general.
- ❖ Incluye soporte para batch, streaming, SQL, machine learning y procesamiento de grafos.
- ❖ Orientado a procesar los datos en **memoria** (en lugar de sólo desde HDFS)
 - Menor **latencia** que Hadoop
- ❖ No tan centrado en **MapReduce**
 - Ofrece **APIs** de alto nivel que incluyen librerías de **Machine Learning**
- ❖ Puede trabajar sobre **HDFS** u otro almacenamiento compatibles con Hadoop como **S3**

- ❖ En pocas palabras, Spark intenta distribuir el trabajo entre la mayor cantidad de workers posible.
- ❖ La mejor parte es que **Spark realiza el paralelismo** por nosotros y sólo tenemos que administrar el clúster y calcular cuántos workers necesitamos para nuestras tareas.
- ❖ Una misma operación puede ejecutarse simultáneamente en cientos o miles de núcleos sin que el programador escriba código de paralelización explícito.

¿Qué es un RDD?



❖ Un RDD (Resilient Distributed Dataset) es una colección distribuida de datos que Spark puede procesar en paralelo.

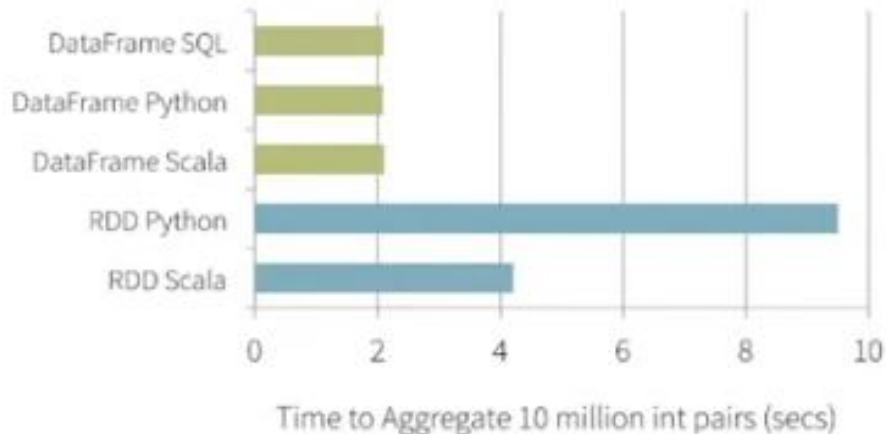
RDDs y Dataframes

RDDs

- Resiliente: tolerante a fallos
- Distribuido: entre múltiple nodos
- Dataset: colección de datos particionados

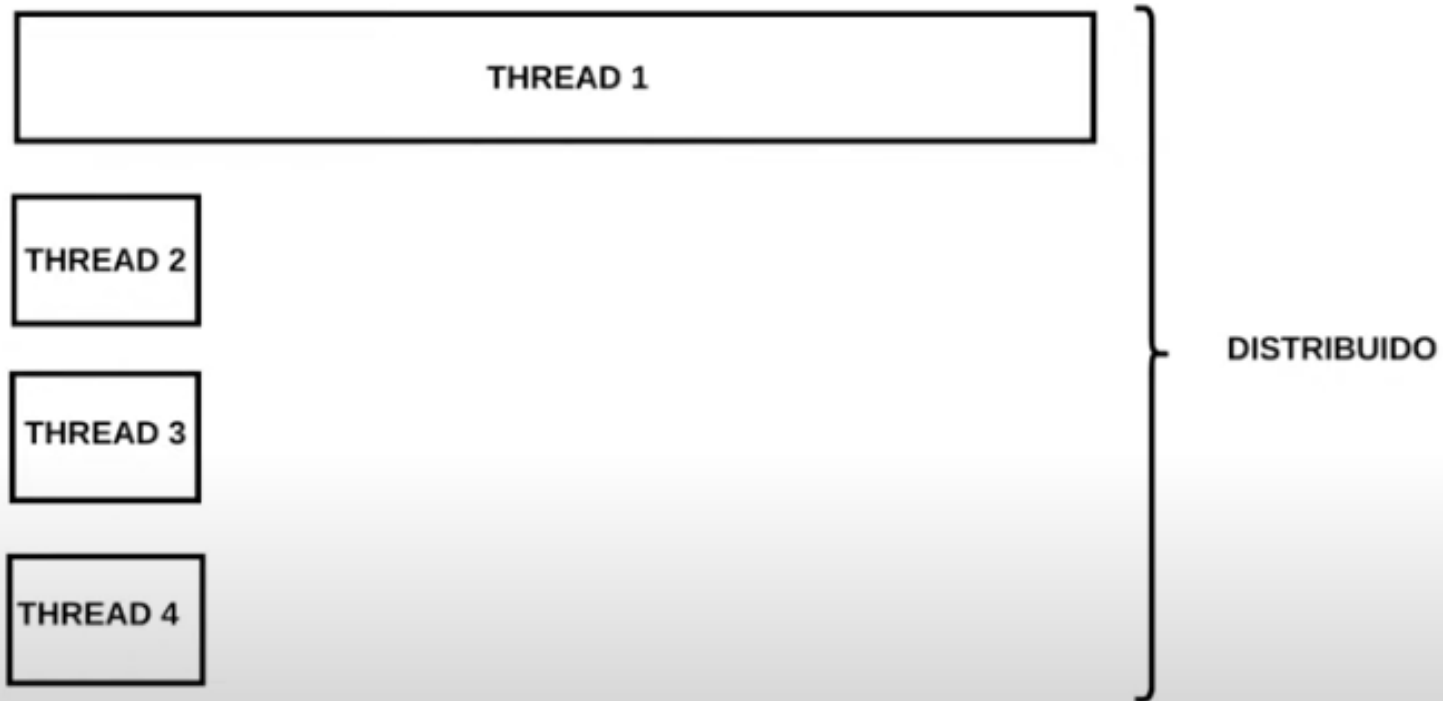
Dataframes

- API superior
- Más sencillos de usar
- Optimizaciones



Data skew

Parallel task



MapReduce vs Spark

— — —



MapReduce

- ❖ Lee desde disco.
- ❖ Procesa.
- ❖ Escribe en disco.
- ❖ Muy robusto.
- ❖ Alta latencia.

Spark

- ❖ Mantiene datos en memoria.
- ❖ Reutiliza resultados intermedios.
- ❖ Menor latencia.
- ❖ Ideal para algoritmos iterativos.

Ejemplos batch processing

Batch processing

— — —



Ejemplos a inspeccionar:

- ❖ **Inverted index: MapReduce vs Spark**

- ❖ **Machine Learning**

- ❖ **Collaborative filtering**

 - **Matrix factorization**

Ejemplos típicos de sistemas de recomendación (Netflix, Spotify, Amazon).

Cierre



Bibliografía

📖 **“Big data: Principles and best practices of scalable realtime data systems”**, N. Marz, J. Warren, 1st edition, 2015.

📌 Capítulo 6: Batch layer

📖 Paper original de MapReduce: **“MapReduce: Simplified Data Processing on Large Clusters”**, J. Dean, S. Ghemawat.
<https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

📖 Cómo implementar correctamente un MapReduce, video de Jeffrey Ullman: <https://www.youtube.com/watch?v=mpCJRq1XruA>